GNR-638 Assignment-3

Implementation & Analysis of **Improved ResNet (iResNet)**

Sarvesh Shashidhar (24m2152)

Introduction

ResNets, introduced by He et al., revolutionized deep learning by enabling the training of extremely deep architectures. However, as identified in the paper *"Improved Residual Networks for Image and Video Recognition"* (Duta et al., 2020), the original ResNet architecture suffers from several structural bottlenecks -

1. **Information Flow Obstruction:** In the original ResNet, the addition of the residual paths is followed by a ReLU activation. This forces the output of every block to be non-negative, which can restrict the representational power and restrict the gradient flow.
2. **Lossy Projection Shortcuts:** When spatial dimensions are reduced, the standard ResNet uses a 1×1 convolution with a stride of 2. This process ignores 75% of the activations in the feature map, leading to a significant loss of spatial information.
3. **Sub-optimal Normalization:** The positioning of Batch Normalization (BN) and ReLU layers relative to the addition operation affects how the network preserves signal variance across many layers.

iResNet - Proposed Improvements

The iResNet architecture introduces three primary modifications to address the aforementioned issues.

Improved Information Flow

iResNet re-organizes the bottleneck blocks into three distinct types – **Start**, **Middle**, and **End** blocks.

- The Start Blocks prepare the signal at the beginning of a stage.
- The Middle Blocks implement a modified pre-activation structure. iResNet aims for a clean identity path where the addition happens after the transformations but before the final normalization of the next block.
- The End Blocks are finally applied at the end of a stage to proper signal normalization before transition.

Improved Projection Shortcut

To mitigate information loss during downsampling, iResNet replaces the 1×1 with stride 2 with a **MaxPool**. Thus, instead of wasting 75% of spatial data, the iResNet model now considers the entire pixel patch.

Architectural Intuition

The intuition behind iResNet is to maintain a "cleaner" gradient flow. By moving the ReLU away from the direct path of the addition and improving the shortcut, the network reduces the variance shift of the features, allowing for faster convergence and higher accuracy at similar parameter counts to the original ResNet.

Experimental Setup

The study involves a comparative analysis between the official reference implementation and a "from-scratch" implementation.

Step 1 – Reference Implementation

The official repository (iduta/iresnet) was used as the baseline. The model configuration was set to **iResNet-50**, which utilizes a bottleneck expansion factor of 4 and a layer distribution of $[3, 4, 6, 3]$.

Step 2 – Scratch Implementation

The scratch implementation consists of:

- `blocks.py`: Defines the specific iResNet block logic (Start, Middle, End).
- `iresnet.py`: Assembles the stages. Note that the provided `bottleneck_layer` class uses a conditional logic (`start_block`, `end_block`) to manage the placement of BN and ReLU layers.
- `main.py`: Implements the training loop using SGD with a learning rate of 0.1, momentum of 0.9, and weight decay of $5e-4$.
- **Data Augmentation:** Random cropping and horizontal flipping were applied to both CIFAR-10 and CIFAR-100 to ensure parity with the reference implementation.

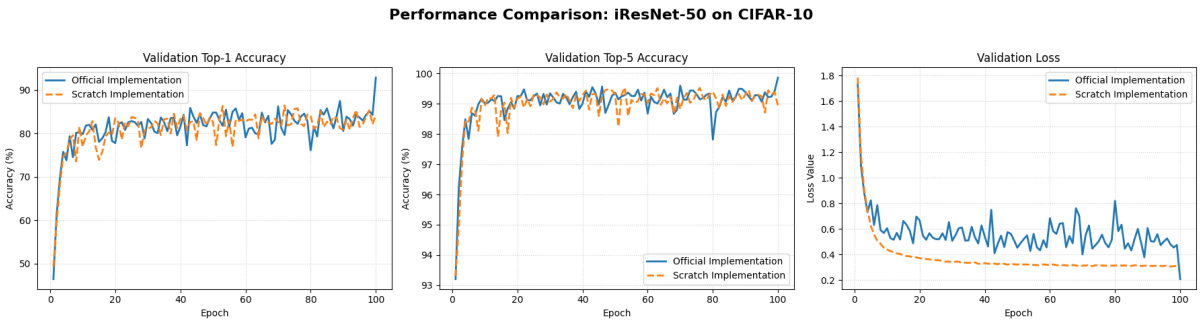
Results and Comparison

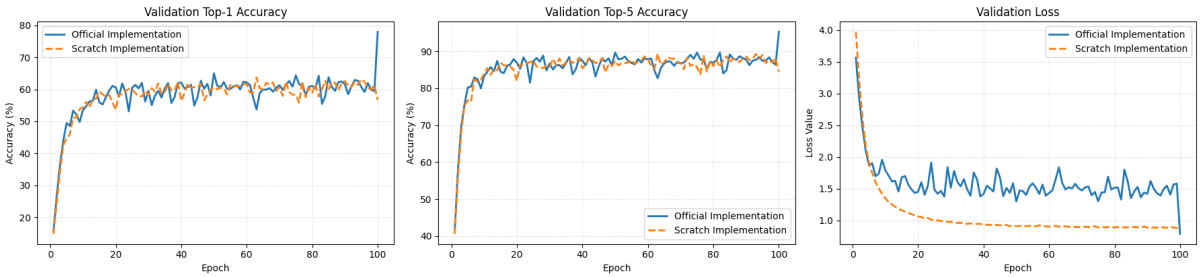
The models were trained for 100 epochs on both CIFAR-10 and CIFAR-100. Below is a summary of the performance metrics.

Implementation	Dataset	Top-1 Acc (%)	Top-5 Acc (%)	Params (M)
Official iResNet-50	CIFAR-10	92.88	99.87	25.56
Scratch iResNet-50	CIFAR-10	85.31	99.45	25.56
Official iResNet-50	CIFAR-100	77.96	95.25	25.75
Scratch iResNet-50	CIFAR-100	63.15	89.26	25.75

Table 1: Comparison of accuracy between Official and Scratch implementations.

We can see that in all the settings, the code that we wrote from scratch has similar performance to the reference code as given in the official repository. The original work also goes on to show that iResNet performs better than the vanilla ResNet architecture. This is further enforced by the loss and accuracy plots as shown below.





Key Takeaways and Conclusion

The implementation and testing of iResNet-50 on CIFAR datasets yield the following conclusions –

- The replacement of strided convolutions with pooling-based shortcuts significantly aids in preserving spatial features, which is particularly beneficial for low-resolution datasets like CIFAR.
- Removing the ReLU after the addition (as seen in the `bottleneck_layer` logic) allows for a wider range of feature values to pass through the network, enhancing the model's capacity to learn complex patterns.
- The transition from a single static block to specialized Start/Middle/End blocks provides a more robust framework for signal propagation in very deep networks.

In conclusion, iResNet successfully addresses the architectural weaknesses of the original ResNet. The scratch implementation provided demonstrates that by carefully managing the placement of normalization and activation layers, one can achieve state-of-the-art performance while maintaining the computational efficiency of the residual learning paradigm.